

Cahier des Charges - Application de Gestion Commerciale

Informations Générales

Titre du projet : Application de Gestion Commerciale

Plateforme cible : Linux (Ubuntu 22.04/24.04, Debian, Fedora)

Type d'application : Desktop cross-platform

Équipe de Développement

- **Développement Backend/Database :** Samir Houssaini
 - **Développement Integration/Full-stack :** Mel Russel Fuedong Djou
 - **Développement Frontend/UI :** Chaïma Hajam
-

Objectif du Projet

Développer une application desktop professionnelle de gestion commerciale permettant de gérer efficacement les clients, les produits et les commandes d'une entreprise. L'application doit fonctionner nativement sur Linux avec une interface moderne et intuitive.

Contexte et Périmètre

Contexte Métier

L'application vise à digitaliser et centraliser la gestion commerciale d'une PME en offrant :

- Un suivi centralisé de la base clients
- Une gestion des stocks de produits
- Un système de commandes avec calcul automatique et mise à jour des stocks
-

Périmètre Fonctionnel

Inclus dans le projet :

- Gestion complète des clients (CRUD)
- Gestion des produits avec suivi des stocks
- Système de commandes avec calcul automatique des montants
- Validation des données saisies
- Interface graphique moderne et responsive
- Persistance des données en base MySQL

Hors périmètre :

- Gestion multi-utilisateurs avec authentification
- Système de facturation
- Exports PDF/Excel
- Statistiques et tableaux de bord avancés
- Application web ou mobile
- Synchronisation cloud

Stack Technique

Technologies Obligatoires

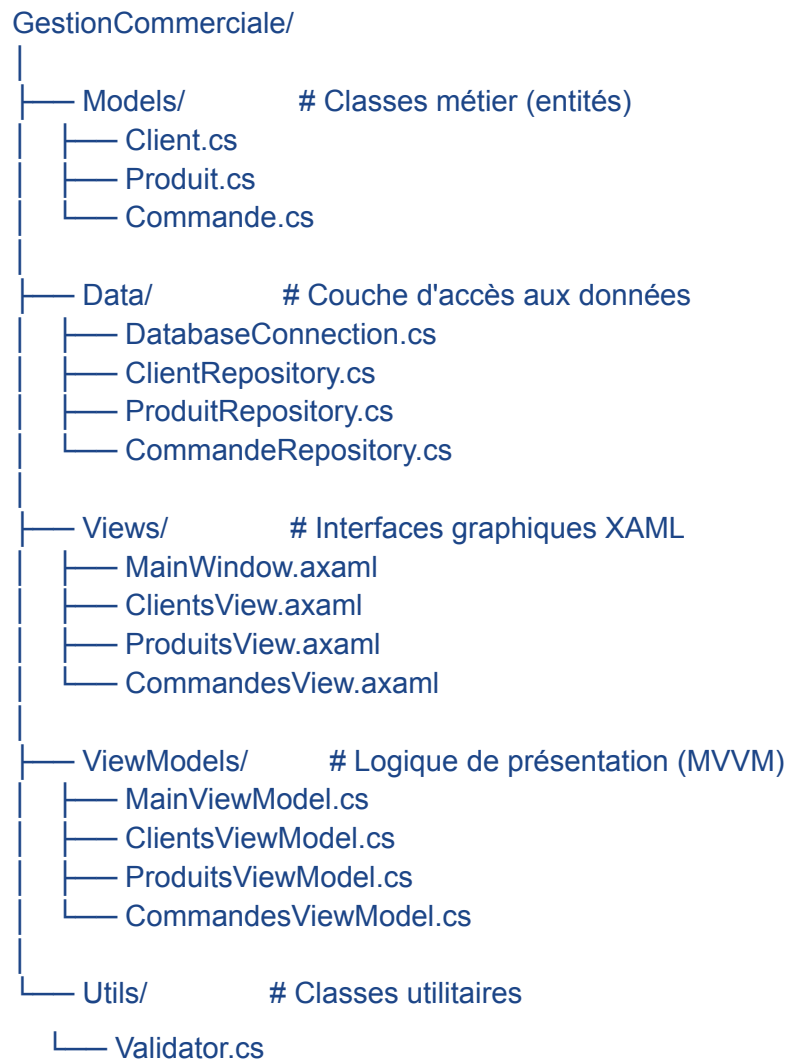
Composant	Technologie	Version
Langage	C#	.NET 8.0
Framework UI	Avalonia UI	Latest
Base de données	MySQL	8.0+
IDE	VS Code	Latest
Pattern architectural	MVVM + Repository	-
Connecteur BDD	MySql.Data	NuGet package

Justification des Choix

- **Avalonia UI** : Framework cross-platform moderne, alternative à WPF pour Linux
- **MySQL** : Base de données relationnelle robuste et gratuite
- **MVVM** : Séparation claire des responsabilités (View/ViewModel/Model)
- **Repository Pattern** : Abstraction de l'accès aux données pour faciliter la maintenance

Architecture de l'Application

Structure des Dossiers

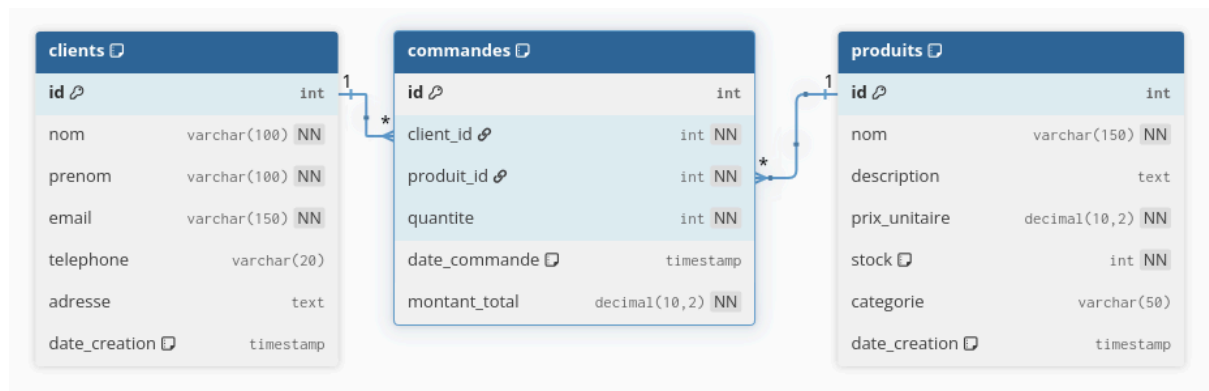


Principes Architecturaux

1. **Séparation des couches** : UI, Logique Métier, Accès Données
2. **MVVM Pattern** : Binding bidirectionnel entre Views et ViewModels
3. **Repository Pattern** : Une classe repository par entité
4. **Validation centralisée** : Classe Validator pour toutes les règles métier
5. **Gestion des ressources** : Utilisation de `using` pour les connexions BDD

Modèle de Données

Schéma Relationnel



Dictionnaire de Données

Table CLIENTS

Champ	Type	Contraintes	Description
id	INT	PK	Identifiant unique
nom	VARCHAR(100)	NOT NULL	Nom de famille
prenom	VARCHAR(100)	NOT NULL	Prénom
email	VARCHAR(150)	UNIQUE, NOT NULL	Adresse email
telephone	VARCHAR(20)	NULL	Numéro de téléphone
adresse	TEXT	NULL	Adresse postale complète
date_creation	TIMESTAMP	DEFAULT NOW()	Date d'ajout

Index : email, nom

Table PRODUITS

Champ	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique
nom	VARCHAR(150)	NOT NULL	Nom du produit
description	TEXT	NULL	Description détaillée
prix_unitaire	DECIMAL(10,2)	NOT NULL, >= 0	Prix en euros
stock	INT	NOT NULL, >= 0, DEFAULT 0	Quantité disponible
categorie	VARCHAR(50)	NULL	Catégorie du produit
date_creation	TIMESTAMP	DEFAULT NOW()	Date d'ajout

Index : nom, categorie

Table COMMANDES

Champ	Type	Contraintes	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique
client_id	INT	FK, NOT NULL	Référence au client
produit_id	INT	FK, NOT NULL	Référence au produit
quantite	INT	NOT NULL, > 0	Quantité commandée
date_commande	TIMESTAMP	DEFAULT NOW()	Date de la commande
montant_total	DECIMAL(10,2)	NOT NULL	Montant total calculé

Index : client_id, produit_id, date_commande

Foreign Keys : ON DELETE CASCADE

Spécifications Fonctionnelles

Module 1 : Gestion des Clients

Fonctionnalités

F1.1 - Affichage de la liste des clients

- **Description** : Afficher tous les clients dans un tableau
- **Données affichées** : Nom complet, Email, Téléphone, Ville
- **Tri** : Par défaut par nom alphabétique
- **Actions** : Sélection d'un client pour modification/suppression

F1.2 - Ajout d'un client

- **Description** : Formulaire de création d'un nouveau client
- **Champs obligatoires** : Nom, Prénom, Email
- **Champs optionnels** : Téléphone, Adresse
- **Validation** :
 - Email au format valide (regex)
 - Email unique (vérification BDD)
 - Téléphone au format français (optionnel)
- **Action** : Enregistrement en base avec message de confirmation

F1.3 - Modification d'un client

- **Description** : Édition des informations d'un client existant
- **Pré-requis** : Sélection d'un client dans la liste
- **Champs modifiables** : Tous sauf ID et date de création
- **Validation** : Identique à l'ajout
- **Action** : Mise à jour en base avec confirmation

F1.4 - Suppression d'un client

- **Description** : Supprimer un client de la base
- **Pré-requis** : Sélection d'un client
- **Confirmation** : Boîte de dialogue "Êtes-vous sûr ?"
- **Cascade** : Suppression automatique des commandes associées
- **Action** : Suppression en base avec confirmation

F1.5 - Recherche de clients

- **Description** : Filtrer la liste par nom ou email
- **Méthode** : Recherche LIKE en BDD ou filtre local
- **Sensibilité** : Insensible à la casse
- **Résultat** : Mise à jour dynamique du tableau

Module 2 : Gestion des Produits

Fonctionnalités

F2.1 - Affichage de la liste des produits

- **Données affichées** : Nom, Description (tronquée), Prix, Stock, Catégorie, Statut
- **Indicateurs visuels** :
 - Stock < 10 : Ligne en orange (alerte stock faible)
 - Stock = 0 : Ligne en rouge (rupture)
- **Tri** : Par défaut par nom

F2.2 - Ajout d'un produit

- **Champs obligatoires** : Nom, Prix unitaire, Stock
- **Champs optionnels** : Description, Catégorie
- **Validation** :
 - Prix >= 0 (décimal à 2 décimales)
 - Stock >= 0 (entier)
 - Nom non vide
 - Description max 500 caractères
- **Action** : Enregistrement avec confirmation

F2.3 - Modification d'un produit

- **Pré-requis** : Sélection d'un produit
- **Champs modifiables** : Tous sauf ID et date de création
- **Alerte** : Si stock < 10 après modification
- **Action** : Mise à jour avec confirmation

F2.4 - Suppression d'un produit

- **Confirmation** : Dialogue avec avertissement si commandes associées
- **Cascade** : Suppression des commandes liées (ou blocage selon choix)
- **Action** : Suppression avec confirmation

F2.5 - Filtrage par catégorie

- **Description** : ComboBox avec liste des catégories
- **Catégories** : Informatique, Accessoires, Audio, Autre
- **Résultat** : Affichage des produits de la catégorie sélectionnée

F2.6 - Calcul de la valeur totale du stock

- **Description** : Afficher la somme (prix × stock) de tous les produits
- **Position** : En bas du tableau
- **Format** : Montant en euros (€)

Module 3 : Gestion des Commandes

Fonctionnalités

F3.1 - Affichage de l'historique des commandes

- **Données affichées** : ID, Nom Client, Nom Produit, Quantité, Montant Total, Date
- **Tri** : Par défaut par date décroissante (plus récentes en premier)
- **Jointures SQL** : Récupération des noms via JOIN

F3.2 - Création d'une commande

- **Étapes** :
 1. Sélection du client (ComboBox avec nom complet)
 2. Sélection du produit (ComboBox avec nom, prix, stock)
 3. Saisie de la quantité
 4. Affichage automatique du prix unitaire et stock disponible
 5. Calcul dynamique du montant total (quantité × prix)
- **Validation** :
 1. Client sélectionné
 2. Produit sélectionné
 3. Quantité > 0
 4. Quantité ≤ stock disponible
- **Contrôle stock** :
 1. Désactivation du bouton si stock insuffisant
 2. Message d'erreur si tentative avec stock = 0
- **Transaction** :
 1. Insertion de la commande
 2. Décrémenter le stock (UPDATE produits)
 3. Commit si succès, Rollback si échec
- **Confirmation** : Message récapitulatif avec détails de la commande

F3.3 - Affichage des détails d'une commande

- **Pré-requis** : Sélection d'une commande dans l'historique
- **Informations** : Toutes les données de la commande
- **Action** : Affichage en lecture seule (pas de modification)

F3.4 - Rafraîchissement de la liste

- **Description** : Recharger l'historique depuis la BDD
- **Utilité** : Voir les nouvelles commandes créées par d'autres utilisateurs (future évolution)

Spécifications Techniques et UI/UX

Charte Graphique

- **Couleur primaire** : Bleu #2563EB
- **Couleur secondaire** : Gris #64748B
- **Couleur d'alerte** : Orange #F59E0B (stock faible), Rouge #EF4444 (rupture)
- **Couleur succès** : Vert #10B981
- **Police** : Segoe UI, Inter ou équivalent Linux

Principes d'Interface

- **Responsive** : Interface adaptable à différentes résolutions (min 1024×768)
- **Cohérence** : Même disposition et style pour tous les modules
- **Accessibilité** : Contraste suffisant, labels clairs
- **Feedback utilisateur** : Messages de confirmation/erreur pour chaque action
- **Espacement** : Marges et paddings cohérents (20px standard)

Fenêtre Principale (Dashboard)

Éléments :

- Titre centré "GESTION COMMERCIALE - DASHBOARD"
- 3 boutons de navigation : "Gérer les Clients", "Gérer les Produits", "Gérer les Commandes"
- Indicateur de connexion BDD : "✓ Connexion réussie" ou "✗ Erreur"

Dimensions : 1000×600 px

Position : Centrée à l'écran

Vues de Gestion (Clients, Produits, Commandes)

Layout type :

- Zone de recherche/filtrage en haut
- DataGrid au centre (liste des éléments)
- Formulaire de saisie en bas ou à droite
- Boutons d'action en bas du formulaire

DataGrid :

- En-têtes clairs et formatés
- Sélection d'une ligne par clic
- Alternance de couleurs de lignes pour la lisibilité
- Scrollbar verticale si nécessaire

Règles de Validation

Validation Côté Client (C#)

Champ	Règle	Message d'erreur
Email	Regex : <code>^[^@\s]+@[^@\s]+\.[^@\s]+\$</code>	"Format d'email invalide"
Téléphone	Format français (optionnel)	"Format de téléphone invalide"
Prix	≥ 0 , format décimal	"Le prix doit être positif"
Stock	≥ 0 , entier	"Le stock doit être positif ou nul"
Quantité	> 0 , entier	"La quantité doit être supérieure à 0"
Quantité vs Stock	Quantité $\leq$ Stock	"Stock insuffisant"
Nom/Prénom	Non vide	"Ce champ est obligatoire"
Description	Max 500 caractères	"Maximum 500 caractères"

Validation Côté Base de Données (SQL)

- **Contraintes CHECK** : prix_unitaire ≥ 0 , stock ≥ 0 , quantite > 0
- **Contraintes UNIQUE** : email des clients
- **Contraintes NOT NULL** : Champs obligatoires
- **Foreign Keys** : Intégrité référentielle avec CASCADE

Plan de Tests

Tests Fonctionnels

Module Clients

Test	Procédure	Résultat attendu
TC-01	Ajouter un client complet	Client enregistré, message de succès
TC-02	Ajouter un client minimal (nom, prénom, email)	Client enregistré
TC-03	Ajouter un client avec email existant	Erreur "Email déjà utilisé"
TC-04	Ajouter un client avec email invalide	Erreur "Format d'email invalide"
TC-05	Modifier un client existant	Modifications enregistrées
TC-06	Supprimer un client sans commandes	Client supprimé
TC-07	Supprimer un client avec commandes	Client ET commandes supprimés (cascade)
TC-08	Rechercher par nom partiel	Liste filtrée correctement

Module Commandes

Test	Procédure	Résultat attendu
TM-01	Créer une commande avec stock suffisant	Commande créée, stock décrémenté
TM-02	Créer une commande avec quantité > stock	Erreur "Stock insuffisant", bouton désactivé
TM-03	Créer une commande avec produit stock = 0	Erreur, impossible de créer
TM-04	Créer une commande puis vérifier le stock	Stock = stock_initial - quantité
TM-05	Vérifier le calcul du montant total	Montant = quantité × prix_unitaire

Module Produits

Test	Procédure	Résultat attendu
TP-01	Ajouter un produit avec stock > 0	Produit enregistré
TP-02	Ajouter un produit avec prix négatif	Erreur "Prix invalide"
TP-03	Ajouter un produit avec stock négatif	Erreur "Stock invalide"
TP-04	Modifier le stock d'un produit	Stock mis à jour
TP-05	Filtrer par catégorie "Informatique"	Seuls produits de cette catégorie affichés
TP-06	Afficher produit avec stock < 10	Ligne en orange
TP-07	Afficher produit avec stock = 0	Ligne en rouge

Tests d'Intégrité

Test	Procédure	Résultat attendu
TI-01	Vérifier les données après ajout	Données correctes dans MySQL Workbench
TI-02	Tester suppression cascade	Commandes supprimées avec le client
TI-03	Tester contrainte UNIQUE email	Insertion refusée si doublon
TI-04	Tester transaction commande	Rollback si erreur, données cohérentes

Livrables

Code Source

1. **Projet C# complet** avec structure organisée
2. **Scripts SQL** :
 - `schema.sql` : Création des tables
 - `data.sql` : Données de test
 - `backup.sql` : Script de sauvegarde
3. **Fichiers de configuration** : `appsettings.json` (optionnel)

Documentation

1. **README.md** :
 - Description du projet
 - Instructions d'installation sur Linux
 - Guide d'utilisation
 - Technologies utilisées
 - Auteurs
2. **docs/database_schema.md** : Schéma de la BDD avec explications
3. **docs/tests.md** : Checklist de tests et résultats
4. **docs/bugs.md** : Bugs connus et résolutions

Visuels

1. **Screenshots** :
 - Dashboard principal
 - Module Clients avec données
 - Module Produits avec alerte stock
 - Module Commandes
 - MySQL Workbench avec données
2. **Mockups** (optionnel) : Design des interfaces avant développement

Dépôt GitHub

- Repository public :
[https://github.com/\[username\]/gestion-commerciale](https://github.com/[username]/gestion-commerciale)
- Branches :
 - `main` : Version stable
 - `develop` : Développement en cours
 - `feature/*` : Fonctionnalités spécifiques
- Commits clairs et réguliers avec messages descriptifs

Planning Prévisionnel

Jour 1 - Setup & Infrastructure (8h)

Tâche	Responsable	Durée
Installation environnement (tous)	Tous	2h
Création base de données MySQL	Samir	3h
Architecture du projet C#	Russel	3h
Design interface principale	Chaïma	3h

Jour 2 - Module Clients (10h)

Tâche	Responsable	Durée
ClientRepository (CRUD)	Samir	4h
Interface ClientsView	Chaïma	4h
Intégration et logique métier	Russel	4h

Jour 3 - Module Produits (10h)

Tâche	Responsable	Durée
ProduitRepository	Russel	3h
Interface ProduitsView	Chaïma	3h
Intégration et filtres	Samir	4h

Jour 4 - Module Commandes (12h)

Tâche	Responsable	Durée
CommandeRepository + Transactions	Russel	4h
Interface CommandesView	Chaïma	4h
Intégration multi-repositories	Samir	4h

Jour 5 - Finitions (6h)

Tâche	Responsable	Durée
Tests complets	Russel	3h
Corrections bugs	Samir	2h
Documentation + GitHub	Chaïma	2h

Total estimé : 46 heures (réparties sur 5 jours avec chevauchements)

Contraintes et Risques

Contraintes Techniques

- **Système d'exploitation** : Linux uniquement (Ubuntu, Debian, Fedora)
- **Framework UI** : Avalonia (pas de WPF, Windows Forms)
- **Base de données** : MySQL 8.0 minimum
- **Connexions BDD** : Gestion stricte avec `using` pour éviter les fuites
- **Transactions** : Obligatoires pour les opérations critiques (commandes)

Contraintes Fonctionnelles

- **Performance** : Chargement de la liste < 2 secondes pour 1000 éléments
- **Validation** : Tous les champs doivent être validés avant insertion
- **Feedback** : Message utilisateur pour chaque action (succès/erreur)
- **Intégrité** : Pas de données orphelines (cascade ou blocage)

Risques Identifiés

Risque	Probabilité	Impact	Mitigation
Problème de compatibilité Avalonia sur certaines distros Linux	Moyen	Élevé	Tester sur plusieurs distros, prévoir VS Code comme alternative à Rider
Complexité des transactions SQL	Moyen	Moyen	Bien documenter le code, tester exhaustivement
Manque d'expérience équipe avec MVVM	Élevé	Moyen	Tutoriels, documentation ReactiveUI, pair programming
Bugs de concurrence sur le stock	Faible	Élevé	Utiliser des transactions SQL, tester les cas limites
Dépassement de délai	Moyen	Moyen	Prioriser les fonctionnalités essentielles, version minimale fonctionnelle d'abord

Critères de Réussite

Le projet sera considéré comme réussi si :

1. L'application compile et s'exécute sur Linux sans erreur
2. La connexion à MySQL fonctionne avec gestion des erreurs
3. Les 3 modules (Clients, Produits, Commandes) sont fonctionnels et complets
4. Toutes les opérations CRUD fonctionnent correctement
5. Les validations empêchent les données invalides
6. Le stock est correctement mis à jour lors des commandes
7. Les transactions garantissent l'intégrité des données
8. L'interface est moderne, cohérente et utilisable
9. Le code est structuré selon MVVM et Repository Pattern
10. Le projet est versionné sur GitHub avec un README complet
11. Au moins 80% des tests fonctionnels passent avec succès
12. La documentation technique est complète et à jour

Support et Ressources

Documentation Officielle

- .NET sur Linux : <https://learn.microsoft.com/dotnet/core/install/linux>
- Avalonia UI : <https://docs.avaloniaui.net/>
- MySQL : <https://dev.mysql.com/doc/>
- ReactiveUI (MVVM) : <https://www.reactiveui.net/>

Outils de Support

- Forum Avalonia : <https://github.com/AvaloniaUI/Avalonia/discussions>
- Stack Overflow : Tags [c#](#), [avalonia](#), [mysql](#), [mvvm](#)
- Discord Avalonia : Communauté active pour questions

Document validé par : Smir Houssaini, Chaïma Hajam, Mel Russel Fuedong Djou

Date de validation : 14/01/2025

Version : 1.0